WOD2Sim: Contract-Based System Integration of Dataset-Trained Driving Policies into Distributed Closed-Loop Simulation

Alba Maria Tellez Fernandez
Independent Researcher
Email: amtellezfernandez@gmail.com

***Abstract*—Closed-loop simulation is useful only when a failure can be assigned to the policy being evaluated rather than to the integration layer that drives the simulator. Dataset-trained driving policies are usually built around logged observations, route geometry or route intent, and fixed-cadence trajectory predictions; a distributed simulator delivers asynchronous messages, expects actions on its own control clock, and depends on runtime preconditions that may be invisible in a benchmark table. WOD2Sim treats this mismatch as a system-integration problem instead of a format conversion problem. It defines a contract-validation layer for WOD-style policies running as AlpaSim external drivers: semantic contracts preserve route and scene meaning, temporal contracts resample trajectories on the simulator grid, lifecycle contracts keep driver sessions explicit and idempotent, deployment contracts materialize prerequisites, and evidence contracts decide when a rollout can support a policy claim. The reference implementation provides dependency-light policies, learned-policy entry points, actor-aware planner hooks, manifests, audits, and aggregation scripts behind one driver interface. The current public release reports contract-validity and failure-attribution evidence, not a claim-valid policy benchmark; rows that violate integration, precondition, or evidence contracts are retained as non-policy-attributed failures.**

I. Introduction

Dataset-trained driving policies and closed-loop simulators expose different failure surfaces. A WOD-style policy consumes logged observations, route intent or route geometry, and a short-horizon ego-relative trajectory target. A closed-loop external driver must instead remain alive as a service, accept incremental sensor and route messages, return controls on the simulator clock, and leave an evidence trail that distinguishes policy behavior from integration failure.

This distinction matters because a wrapper can make integration failures look like policy failures. For example, reducing route geometry to a high-level command can remove the local polyline used by a route-conditioned trajectory policy. A later collision, route drift, or invalid trajectory may then be attributed to policy quality even though the decision-relevant input was lost at the simulator boundary. Similar confounds arise from mismatched trajectory sampling grids, stale observations, duplicate close messages, optional backend imports, missing scene assets, and audit artifacts that cannot support scientific claims.

WOD2Sim addresses this problem as system integration. It provides a reference boundary for running heterogeneous trajectory policies through AlpaSim's external-driver interface while making each boundary assumption inspectable. The central claim is not that WOD2Sim is a new driving policy or an official WOD benchmark implementation. The claim is that dataset-trained policies require explicit semantic, temporal, lifecycle, deployment, and evidence contracts before closed-loop results can be interpreted.

This paper asks two questions: which contract mismatches prevent dataset-trained policies from operating as reliable simulator services, and can explicit contract validation distinguish integration and runtime failures from policy behavior failures? The current repository does not yet support a complete claim-valid benchmark because direct actor-aware rows remain blocked by a missing scene-matched oracle proxy and learned checkpoints are not part of the public release. It does provide a reproducible evidence package whose completed, planned, blocked, and failed rows are traceable to repository artifacts.

A. Failure Attribution Rule

WOD2Sim uses an explicit attribution rule before reporting any behavior or failure metric. A rollout event may be interpreted as policy behavior or policy failure only when the semantic route contract, temporal adapter, lifecycle state, deployment preconditions, and evidence audit all pass. If any of those gates fails, the row is classified as an integration, precondition, or evidence failure. A missing actor proxy is therefore not a weak planner, a command-proxy route is not a bad route-conditioned policy, and an incomplete manifest is not a collision result. Passing all gates permits policy-behavior attribution, not automatic policy-failure attribution; policy failure additionally requires the retained failure layer to be policy. This rule is intentionally stricter than ordinary demo reporting because the scientific risk is misattributing adapter or runtime faults to the policy being integrated. WOD2Sim therefore evaluates integration validity first and policy quality only after the row is claim-valid.

II. Integration Problem and Requirements

The offline-policy contract and the external-driver contract differ along five layers. Table I summarizes the recurring mismatches and the WOD2Sim mechanisms used to expose them.

TABLE I
CONTRACT MAP FOR INTEGRATING WOD-STYLE TRAJECTORY POLICIES INTO DISTRIBUTED CLOSED-LOOP SIMULATION.

Mismatch	Contract	Mechanism	Validation
Command-only route	Semantic	Preserve route geometry	route-source audit
Policy horizon/runtime grid	Temporal	Deterministic resampling	cadence tests
Script flow/session service	Lifecycle	Idempotent late-event handling	lifecycle tests
Implicit host state	Deployment	Materialized manifests	readiness checks
Process exit/evidence	Evidence	Audit-valid summaries	claim gate

A. Semantic Contract

The semantic contract preserves decision-relevant meaning across the dataset-policy and simulator boundary. Route geometry must reach prediction time as route geometry; a left/straight/right command remains useful context but cannot replace a local polyline. The policy-facing route must follow the ego travel lane rather than an unrelated road center. Ego history, speed, acceleration, camera availability, static hazards, and dynamic actors must retain consistent geometry and motion fields where those fields are available. Visibility diagnostics must not fabricate obstacles, and command-proxy route fallbacks must be labeled diagnostic rather than claim-valid.

B. Temporal Contract

The temporal contract aligns policy outputs with the runtime clock. Source and target horizons are explicit, timestamps are monotonic, and the current ego origin anchors interpolation. Matching grids preserve the trajectory, while mismatched grids are resampled deterministically and headings are recomputed on runtime points. Non-finite, malformed, or structurally invalid trajectories must be rejected or handled safely.

C. Lifecycle Contract

The lifecycle contract turns a policy into a persistent service. Session creation, active state, close, cleanup, and duplicate close are explicit. Late image, egomotion, route, and close events are counted and traced instead of crashing the service. Session state must not leak across repeated or interleaved sessions, and fatal errors must be distinguished from recoverable lifecycle warnings.

D. Deployment and Evidence Contracts

The deployment contract materializes the run: driver and wizard commands, topology, ports, scene identifiers, timeout, policy configuration, external-driver address, simulator Git state, patch state, Docker image identity, GPU state, and checkpoint hashes where applicable. The evidence contract defines when a rollout can support a paper claim: unique run identifiers, immutable manifests, terminal status, route-source and sensor-freshness validity, conformance checks, audits, support-bundle status, failed-run retention, and hashes tying generated numbers to artifacts.

III. WOD2SIM ARCHITECTURE

Figure 1 shows the system boundary. WOD2Sim sits between an offline or dataset-trained trajectory policy and the distributed AlpaSim runtime. The adapter reconstructs policy-facing state, preserves route geometry, resamples outputs, and isolates model discovery from unrelated optional backends. The runtime side records deployment state and evidence artifacts before any claim is accepted.

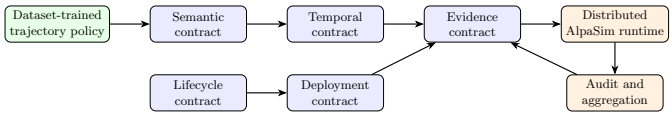


Fig. 1. WOD2Sim localizes integration failures at contract boundaries between a WOD-style policy and a distributed AlpaSim runtime.

The implementation registers dependency-light baselines and learned-policy adapter entry points through AlpaSim model discovery. Constant-velocity and route-following baselines do not require learned checkpoints. Token BC/DAgger and direct actor-aware planning entry points are present, but their execution requires the relevant local checkpoint or scene-matched actor-proxy artifacts. The current CVM therefore treats missing direct-actor proxy state as a deployment blocker, not as a policy failure.

IV. IMPLEMENTATION AND EVALUATION METHOD

The CVM package uses deterministic configuration files for core, semantic ablation, temporal ablation, lifecycle stress, and fault-injection matrices. The core matrix is configured for three policies, six locally cached front-camera scenes, and three replicate identifiers. These identifiers are provenance labels in the current launcher; AlpaSim runtime seeds are recorded as execution metadata when exposed, but the configured identifier is not yet a simulator seed override. The semantic and temporal ablations each configure paired route or resampling variants on three scenes and three replicate identifiers. Lifecycle stress configures repeated service-harness cycles, and the fault matrix enumerates semantic, temporal, lifecycle, deployment/plugin-dependency, and evidence injections.

Scene selection is intentionally conservative. Six scenes are selected from available local 26.02 front-camera USDZ artifacts, but the repository does not currently provide authoritative metadata proving straight, intersection, lane-change, dense-traffic, occlusion, or merge categories. The manifest therefore records them as available but unclassified. Each run manifest carries this `scenario_category`, asset availability, selection rationale, route/interaction feature expectations, and license-gating status; the paper does not claim scenario-category coverage.

The evaluation runner expands matrices into rows, writes CSV and JSON summaries, and returns nonzero status when any row is not completed. Aggregation preserves failed and blocked rows, separates planned rows from blockers, rejects duplicate completed run identifiers, writes LaTeX tables with data hashes, and generates vector figures. In the current environment, public synthetic lifecycle and fault-injection harness rows execute, and any completed closed-loop rows are audited for route-source and sensor-freshness validity before they are counted as diagnostic rollout evidence. The

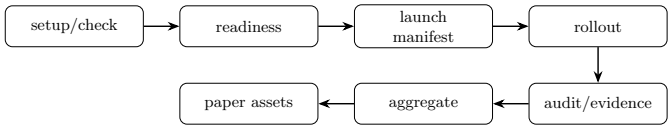


Fig. 2. The contract-validation pipeline separates setup, readiness, manifest materialization, rollout, audit, aggregation, and paper-asset generation.

resulting rows document the exact experiment denominator while preventing unexecuted protocols from becoming results.

V. Results

No complete policy benchmark is claim-valid in the current repository state. Table II reports the generated core policy status and the closed-loop diagnostic metrics currently available for dependency-light policies.

The configured matrices contain 145 rows: 54 core rows, 18 semantic-ablation rows, 18 temporal-ablation rows, 40 lifecycle rows, and 15 fault-injection rows. The current aggregate attempts 109 rows and completes 109 rows, including 54 closed-loop rollout(s). Among completed full-contract rollouts, 45/ 45 pass the route and sensor audit. The false-block check currently observes 0/ 45 valid full-contract rows incorrectly blocked by the evidence gate. Matched semantic-confound evidence includes 9/9 metric-bearing pairs. Across those pairs, full-contract minus command-only route means are -0.243 progress, 0.007 relative progress, 0.333 collision-any, 0.000 off-road, and 0.353 plan deviation. These deltas show that losing route geometry changes measured rollout behavior, but they are not a policy-quality comparison: all 9 command-only rows are rejected by the audit because they use command-proxy route evidence. The aggregate keeps 0 rows as planned/not launched and 36 rows as true precondition or unsupported-ablation blockers.

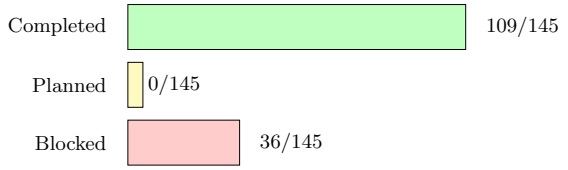
The public synthetic diagnostics in Table III show that the hardened lifecycle harness survives 20/20 cycles, while the strict/pre-hardening comparison survives 0/20 cycles. The fault harness detects 15/15 configured injections and localizes 15/15 to the expected contract layer/code. These are diagnostic harness results, not policy-performance or real-scene reliability metrics.

The current effectiveness result is therefore bounded. Completed full-contract rollouts measure whether valid integrations pass the audit without false blocking, while matched semantic-ablation rows measure a real route-loss confound and show that the evidence gate prevents command-only route rows from becoming claim-valid policy evidence. The synthetic lifecycle and fault counts are retained as diagnostics only and are not used as evidence that the integration method is faster, safer, or more correct than a functional non-contract wrapper.

The same aggregate also reports failure attribution. It contains 45 contract-valid closed-loop row(s), 9 completed closed-loop row(s) rejected as integration/evidence-invalid, 36 precondition-blocked row(s), 36 integration-failure-attributable row(s), 55 synthetic diagnostic row(s), and 0 claim-valid policy benchmark row(s). These counts are the operational separation between

TABLE II
Core policy matrix status. Rows are configured core rows; Done/att. gives completed/attempted launches, Audit gives audit-valid completions, Lat. p95 is reported only when frame-level latency evidence is available, Crash counts terminal service-crash rows, and direct actor rows remain deployment-blocked.

Policy	Rows	Done/att.	Audit	Progress	Coll./off	Lat. p95	Crash	Blocked
constant_velocity	18	18/18	18/18	0.435	0.833/0.000	–	0	0
direct_actor_planner	18	0/0	0/0	–	–/–	–	0	18
route_following	18	18/18	18/18	0.353	0.833/0.000	–	0	0



Configured CVM status; blocked rows keep true blockers visible.

Fig. 3. Configured CVM status. The denominator is 145 configured rows; planned rows, if any, are not launched, and blocked rows name true precondition or ablation-support gaps.

integration failure and policy failure in the current release: 0 row(s) are policy-behavior attributable, 0 row(s) are policy-failure attributable, and 145 row(s) remain non-policy-attributed. No blocked, contract-invalid, or diagnostic row can assign policy failure, even when its behavior metrics are degraded. A policy failure also requires the retained failure layer to be policy, not merely a degraded behavior metric.

VI. Discussion, Limitations, and Related Work

The framework generalizes beyond a single simulator because the contracts identify boundary assumptions rather than AlpaSim-specific filenames. The five contracts are an engineering instantiation of contract-based cyber-physical system design and interface-level reasoning [1], [2]: an offline policy assumes route, timing, lifecycle, deployment, and evidence conditions, while the simulator boundary must either guarantee them or reject the run as non-claim-valid. Route geometry, sampling cadence, lifecycle robustness, deployment provenance, and claim-valid evidence also matter in WOD-style datasets [3] and in closed-loop planners and simulators such as nuPlan [4], CARLA [5], Waymax [6], and NAVSIM [7]. Scenario and falsification tools such as Scenic, VerifAI, DriveFuzz, and PlanFuzz [8], [9], [10], [11] exercise autonomous-driving systems with generated or mutated conditions; WOD2Sim is complementary because it gates whether the simulator/policy boundary itself preserved the assumptions needed to interpret a rollout. WOD2Sim uses AlpaSim as the case study because its external-driver boundary makes these contracts explicit.

The limitations are material. WOD2Sim is not an official Waymo challenge interface, does not reconstruct complete WOD worlds, and does not redistribute restricted scene assets. The current CVM package has no completed claim-valid benchmark matrix, no validated scene-category coverage, and no scene-matched direct-actor proxy for direct actor-aware

TABLE III
PUBLIC SYNTHETIC DIAGNOSTICS. THESE ROWS EXERCISE SERVICE-LEVEL HARNESSES ONLY; THEY ARE NOT CLOSED-LOOP SCENE ROLLOUTS.

Synthetic diagnostic	Count	Total
Lifecycle hardening survived	20	20
Pre-hardening survived	0	20
Faults detected	15	15
Faults localized	15	15

planning. Its lifecycle and fault-injection evidence comes from public synthetic harnesses, not simulator-backed stress trials. Learned token policies remain outside the public baseline unless a legitimate local checkpoint and hash are provided.

These limitations also explain why the paper does not compare policy quality. A framework paper can report reliability, validity, and diagnostic coverage only after the corresponding rows are executed and audited. Until then, the strongest supported claim is that the repository materializes and blocks incomplete evidence rather than allowing it to appear as a benchmark.

VII. CONCLUSION

Integrating a dataset-trained trajectory policy into distributed closed-loop simulation requires more than input/output conversion. WOD2Sim formulates the boundary as five contracts: semantic, temporal, lifecycle, deployment, and evidence. The current repository implements the contract-oriented release surface and a reproducible CVM evidence pipeline. The generated CVM aggregate separates completed/audit-valid rollout evidence, planned rows, true blockers, and synthetic diagnostics, but it still reports no full claim-valid benchmark. That separation is the useful integration outcome: it prevents route, timing, lifecycle, deployment, and evidence failures from being misreported as policy failures until the complete audited matrices exist.

APPENDIX
WOD2SIM REPOSITORY EVIDENCE MAP

The repository and paper share one release surface. The canonical manuscript is the root-level PDF. The paper source, bibliography, generated tables, figure inputs, matrix configuration, row manifests, aggregate CSV and JSON files, and release reports are all kept in named repository directories and rebuilt by the top-level release targets. Obsolete manuscript sources were removed from the release tree so the root PDF and `paper/cvm` source form the single paper surface.

Repository-native code provides the implementation evidence. Simulator adapters implement the semantic and temporal contracts; CLI commands materialize launch, readiness, audit, support-bundle, and reproduction steps; aggregation regenerates every number, table, and figure used in the build from retained matrix evidence.

This appendix is a traceability map, not an additional result. The current root PDF, README, reports, and generated evidence all preserve the same claim boundary: synthetic diagnostics are public service-harness evidence, while closed-loop scene rows are not claim-valid until executed, audited, and retained in the aggregate denominators.

The public repository intentionally separates source, execution evidence, and paper assets. The implementation layer contains the simulator adapters, route and scene-signal conversion, temporal resampling, plugin entry points, setup/readiness commands, audit logic, and support-bundle generation. The evaluation layer contains the CVM configuration, scene manifest, row manifests, retained CSV/JSON summaries, blocked-run reports, and generated table and figure assets. The manuscript layer consumes only those generated artifacts; manual edits to tables are not part of the release workflow.

The temporal adapter is traceable at unit-test level. Runtime frequency and horizon are validated as positive finite values; trajectories must be finite $(N, 2)$ arrays; the current ego pose is the interpolation origin; optional source timestamps must be finite, strictly increasing, and inside the configured horizon; and headings are recomputed on the resampled runtime grid. These tests support the temporal contract statement, but they do not substitute for the blocked temporal-ablation scene matrix.

The three paper figures serve different audit roles. The architecture figure identifies where semantic, temporal, lifecycle, deployment, and evidence failures cross the external-driver boundary. The evaluation-pipeline figure shows how readiness, launch manifests, rollout status, audit, aggregation, and paper assets are chained. The results figure is deliberately a denominator plot: it shows configured, completed, planned, and blocked rows rather than decorating an unavailable benchmark result.

The release package also records what is absent. Gated scenes, private credentials, learned checkpoints, and proprietary container layers are not bundled. When they are needed for a future rollout, the repository records identifiers, hashes, commands, and precondition states so that restricted material does not become part of the public release and missing inputs are not misreported as policy behavior.

The intended reading order for the public release is therefore not only the paper. The repository inventory records the source tree, environment, build commands, and known gated prerequisites. The contract-test audit maps each required semantic, temporal, lifecycle, deployment/plugin-dependency, evidence, and fault-injection behavior to executable tests or to an explicit gap. The claim-evidence matrix is the final gate: a statement may appear as a paper claim only when it names the producing command, test, and evidence that support it. The blockers report keeps unavailable closed-loop rows visible, while planned rows remain visible without being misclassified as infrastructure blockers.

This structure is deliberately stricter than a demo-first release. The synthetic demo proves that command materialization, audit generation, support-bundle construction, and public plotting work without restricted assets. It does not prove closed-loop

policy performance. Conversely, the matrix runner can create manifests for closed-loop scene rows before launch, preserving scene identifiers, policy names, replicate identifiers, scenario categories, configuration hashes, runtime seed metadata when available, and precondition states. That makes a future rerun additive: once assets, actor proxies, ablation flags, and runtime dependencies exist, new completed rows can be aggregated beside the retained planned and blocked rows rather than replacing them.

The release commands follow the same separation. Inventory captures repository and environment state; check runs lint, conformance, and submission validation; demo generates a public synthetic run; synthetic evaluation executes lifecycle and fault-injection harnesses; scene evaluation records completed, planned, and blocked rows; aggregation regenerates result tables and figures; the paper target rebuilds the sole root-level PDF; and validation checks page count, A4 size, metadata, generated-artifact hashes, unresolved references, forbidden placeholder text, private paths, credentials, and duplicate manuscript PDFs. These gates are engineering controls, not scientific results.

REFERENCES

[1] A. Sangiovanni-Vincentelli, W. Damm, and R. Passerone, "Taming dr. frankenstein: Contract-based design for cyber-physical systems," *European Journal of Control*, vol. 18, no. 3, pp. 217–238, 2012.

[2] L. de Alfaro and T. A. Henzinger, "Interface automata," in *Proceedings of the 8th European Software Engineering Conference*, pp. 109–120, ACM, 2001.

[3] P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, *et al.*, "Scalability in perception for autonomous driving: Waymo open dataset," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2446–2454, 2020.

[4] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. Wolff, A. Lang, L. Fletcher, O. Beijbom, and S. Omari, "nuPlan: A closed-loop ml-based planning benchmark for autonomous vehicles," in *CVPR Workshop on Autonomous Driving*, 2021.

[5] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning*, vol. 78 of *Proceedings of Machine Learning Research*, pp. 1–16, PMLR, 2017.

[6] C. Gulino, J. Fu, W. Luo, G. Tucker, E. Bronstein, Y. Lu, J. Harb, X. Pan, Y. Wang, X. Chen, J. D. Co-Reyes, R. Agarwal, R. Roelofs, Y. Lu, N. Montali, P. Mougin, Z. Yang, B. White, A. Faust, R. McAllister, D. Anguelov, and B. Sapp, "Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research," in *Advances in Neural Information Processing Systems*, 2023.

[7] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta, "NAVSIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking," in *Advances in Neural Information Processing Systems*, 2024.

[8] D. J. Fremont, T. Dreossi, S. Ghosh, X. Yue, A. L. Sangiovanni-Vincentelli, and S. A. Seshia, "Scenic: A language for scenario specification and scene generation," in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp. 63–78, ACM, 2019.

[9] T. Dreossi, D. J. Fremont, S. Ghosh, E. Kim, H. Ravanbakhsh, M. Vazquez-Chanlatte, and S. A. Seshia, "VerifAI: A toolkit for the formal design and analysis of artificial intelligence-based systems," in *Computer Aided Verification*, pp. 432–442, Springer, 2019.

[10] S. Kim, M. Liu, J. Rhee, Y. Jeon, Y. Kwon, and C. H. Kim, "DriveFuzz: Discovering autonomous driving bugs through driving quality-guided fuzzing," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ACM, 2022.

[11] Z. Wan, J. Shen, J. Chuang, X. Xia, J. Garcia, J. Ma, and Q. A. Chen, "Too afraid to drive: Systematic discovery of semantic DoS vulnerability in autonomous driving planning under physical-world attacks," in *Proceedings of the Network and Distributed System Security Symposium*, Internet Society, 2022.